

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Факультет информационных систем и технологий
Кафедра «Измерительно-вычислительные комплексы»

**Отчет по лабораторной работе №5
по дисциплине «Системы искусственного интеллекта»**

Выполнил:
студент группы ИСТбд-31 Коняхин Н.С

Проверил:
Преподаватель Шишкин В.В.

Ульяновск, 2026

Цель:

Разработать алгоритм ElasticNet, MLPRegression в самописном варианте и сравнить с работой модели из библиотеки SCikit-learn. Рассмотреть работу на разном наборе данных

1. Выбор датасета и целевого столбца

Датасет использован по заданию от преподавателя. Целевым столбцом выбрал цену так как эта величина наиболее интересна для предсказания в датасете

2. Выбор количества признаков

ДЛЯ MLPregression

Скрытые слои [128, 64] – два слоя дают достаточную нелинейную ёмкость для табличных данных квартирных цен, не приводя к сильному переобучению при умеренном размере датасета. Ширина 128 → 64 – плавное сжатие информации к выходному слою, часто используемое на практике.

- **Активация relu** – стандартный выбор для внутренних слоёв MLP: ускоряет обучение, не страдает от затухания градиентов в сравнении с сигмоидой/tanh.
- **Скорость обучения 0.001** – типичное начальное значение для Adam, обеспечивает стабильную сходимость без расходимости, особенно после масштабирования признаков.
- **Размер батча 64** – компромисс между шумом градиента и вычислительной эффективностью; хорошо работает на большинстве задач регрессии с сотнями/тысячами примеров.
- **Число эпох 5000** – избыточное значение, фактическая остановка происходит значительно раньше благодаря **early stopping** (обычно 200–500 эпох). Запас гарантирует, что модель успеет сойтись.
- **validation_split=0.1, patience=50** – отложенные 10% данных для контроля переобучения; терпение 50 эпох даёт небольшой запас после выхода на плато, избегая преждевременной остановки.
- **weight_decay=1e-4** – умеренная L2-регуляризация, слегка штрафующая большие веса и улучшающая обобщение, особенно при небольшом количестве признаков.

ДЛЯ ElasticNET

идеальные значения для алгоритма подбираются по ходу выполнения.

alpha (общая сила регуляризации)

Взяты значения на логарифмической шкале от очень слабой (0.001) до сильной (1.0) регуляризации. После стандартизации признаков коэффициенты обычно лежат в пределах $\pm$ нескольких единиц, поэтому такой диапазон покрывает как почти нерегуляризованную

модель, так и модель с существенным сжатием весов. Редко требуется $\alpha > 1$, если данные нормализованы и нет экстремальной мультиколлинеарности.

1. `l1_ratio` (баланс L1 и L2)

Выбран ряд `[0.1, 0.5, 0.7, 0.9, 0.99]`, чтобы протестировать:

- Почти чистый Ridge (`0.1`) – полезен, когда важны все признаки.
- Сбалансированные варианты (`0.5, 0.7`) – компромисс между отбором признаков и стабильностью.
- Сильный L1-уклон (`0.9, 0.99`) – для разреженных решений, когда многие признаки могут быть нерелевантны.

Крайние значения 0 и 1 не включены осознанно: чистый Ridge и чистый Lasso являются частными случаями, а сетка и так достаточно плотная, чтобы найти эффективное соотношение.

3. Обучение модели MLRegression и ElasticNet на самописной реализации алгоритма

Обучим модель MLRegression на сырых данных, затем на подготовленных используя два метода формирования тестового и тренировочного набора. Первый это «обычное разбиение», второй «кроссвалидация».

```
=====
СЦЕНАРИЙ 1: ОБУЧЕНИЕ И ПРОВЕРКА НА ВСЕХ ДАННЫХ (переобучение)
R² на всех (тренировочных) данных: 0.8074

=====
СЦЕНАРИЙ 2: ОБЫЧНОЕ РАЗБИЕНИЕ 80% train / 20% test
Epoch 0: val_loss=0.268655
Epoch 100: val_loss=0.030272
Early stop at epoch 153
R² на тестовой выборке: 0.7966

=====
СЦЕНАРИЙ 3: КРОСС-ВАЛИДАЦИЯ (5 фолдов)

Fold 1/5
R² fold 1: 0.8040

Fold 2/5
R² fold 2: 0.7863

Fold 3/5
R² fold 3: 0.7625

Fold 4/5
R² fold 4: 0.7968

Fold 5/5
R² fold 5: 0.7966

=====
Среднее R² (кросс-валидация): 0.7893 ± 0.0145
Сравнение:
- R² на всех данных (переобучение): 0.8074
- R² на тесте (80/20): 0.7966
- R² среднее по CV: 0.7893
```

Проанализировав данные можно сделать вывод. Что на сырых данных модель переобучилась и выдала нереалистично высокую точность. Обычное распределение дает более точную оценку . кроссвалидация за 5 подходов в среднем дало чуть меньший результат чем обычное разбиение.

```
Подбор гиперпараметров на валидации...
Лучшие гиперпараметры: alpha=0.01, l1_ratio=0.9

=== Сценарий 1: Обучение и оценка на всех данных ===
R² на всех данных (переобучение): 0.6590
MAE на всех данных: 4,545,364 руб.

=== Сценарий 2: Обычное разделение 80/20 ===
R² на тесте (80/20): 0.6772
MAE на тесте: 4,504,773 руб.

=== Сценарий 3: 5-кратная кросс-валидация ===
Фолд 1: R² = 0.5912
Фолд 2: R² = 0.6089
Фолд 3: R² = 0.5971
Фолд 4: R² = 0.5846
Фолд 5: R² = 0.6784

Среднее R² по кросс-валидации: 0.6121 ± 0.0341
```

На данных ElasticNet видно какие параметры были подобраны .На сырых данных модель дала не лучший результат по сравнению с остальными методами. Обычное разбиение показало хороший результат .Кросс валидация в среднем худший результат ,но последний прогон лучший из всех методов.

4. Использование библиотеки SCIKIT-learn

Повторим ранее проведенный эксперимент с MPLRegression реализацией

```

=====
СЦЕНАРИЙ 1: ОБУЧЕНИЕ НА ВСЕХ ДАННЫХ, ПРОВЕРКА НА НИХ ЖЕ
R2 на всех (тренировочных) данных: 0.6351

=====
СЦЕНАРИЙ 2: ОБЫЧНОЕ РАЗБИЕНИЕ 80% train / 20% test
R2 на тестовой выборке: 0.8139

=====
СЦЕНАРИЙ 3: КРОСС-ВАЛИДАЦИЯ (5 фолдов)
R2 по фолдам: [0.92839882 0.93704965 0.93590713 0.93866983 0.93457223]
Среднее R2 (CV): 0.9349 ± 0.0035

=====
Сравнение:
- Переобучение (все данные): R2 = 0.6351
- Hold-out 80/20: R2 = 0.8139
- Кросс-валидация: R2 = 0.9349 ± 0.0035

Process finished with exit code 0

```

Из данных можно заметить что результат. Отличается ,на сырых данных модель недообучилась,на обычном распределении оценка наиболее реалистичная , кросс валидация дала переобучение модели.

```

=== Сценарий 1: Все данные ===
Лучшие параметры: alpha=0.0010, l1_ratio=0.10
R2 на всех данных: 0.6865
MAE на всех данных: 4,340,519 руб.

=== Сценарий 2: Разделение 80/20 ===
Лучшие параметры: alpha=0.0010, l1_ratio=0.10
R2 на тесте: 0.6676
MAE на тесте: 4,447,840 руб.

=== Сценарий 3: Кросс-валидация (5 фолдов) ===
R2 по фолдам: [0.87445554 0.82764184 0.86730483 0.7886203 0.86546403]
Среднее R2: 0.8447 ± 0.0324

```

На данных из ElasticNet библиотеки scikit-learn результат схож но отличается кросс-валидация которая дала высокую точность модели

Вывод:

Выполнив данную работу по обучению модели ElasticNet , MLPRegrettion на алгоритме в двух реализациях использовав несколько методов подготовки данных. Можно сделать вывод о необходимости подготавливать перед обучением модели ,что избежать ее переобучения и повысить ее точность.

ElasticNet самописный

```
import pandas as pd
import numpy as np

# -----
# 1. Загрузка и подготовка данных
# -----
data = pd.read_excel('ml_moscow_flats.xlsx')
data = data.drop_duplicates().reset_index(drop=True)

# Исправление десятичных запятых
for col in ['latitude', 'longitude', 'totalArea', 'kitchenArea']:
    if data[col].dtype == object:
        data[col] = data[col].astype(str).str.replace(',', '.').astype(float)

# -----
# 2. Генерация признаков (такая же, как в исходном решении)
# -----
def haversine_distance(lat1, lon1, lat2, lon2):
    R = 6371
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat/2)**2 + np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2)**2
    c = 2 * np.arcsin(np.sqrt(a))
    return R * c

center_lat, center_lon = 55.751244, 37.618423
data['dist_to_center'] = data.apply(
    lambda row: haversine_distance(row['latitude'], row['longitude'], center_lat, center_lon),
    axis=1)

data['floor_ratio'] = data['floorNumber'] / data['floorsTotal']
data['is_first_floor'] = (data['floorNumber'] == 1).astype(int)
data['is_last_floor'] = (data['floorNumber'] == data['floorsTotal']).astype(int)
data['kitchen_area_ratio'] = data['kitchenArea'] / data['totalArea']
data['total_area_sq'] = data['totalArea'] ** 2
data['log_total_area'] = np.log(data['totalArea'])
data['log_kitchen_area'] = np.log(data['kitchenArea'] + 1)

walls_dummies = pd.get_dummies(data['wallsMaterial'], prefix='walls')
data = pd.concat([data, walls_dummies], axis=1)

features = [
    'floorNumber', 'floorsTotal', 'totalArea', 'kitchenArea',
    'dist_to_center', 'floor_ratio', 'is_first_floor', 'is_last_floor',
```

```

    'kitchen_area_ratio', 'total_area_sq', 'log_total_area', 'log_kitchen_area'
] + list(walls_dummies.columns)

X = data[features].values.astype(np.float64)
y_orig = data['price'].values.astype(np.float64)
y_log = np.log(y_orig)

# -----
# 3. Класс ElasticNetGD и метрика R²
# -----
class ElasticNetGD:
    def __init__(self, alpha=0.1, l1_ratio=0.5, learning_rate=0.01, n_iter=1000, tol=1e-4):
        self.alpha = alpha
        self.l1_ratio = l1_ratio
        self.lr = learning_rate
        self.n_iter = n_iter
        self.tol = tol
        self.weights = None

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        prev_loss = float('inf')

        for _ in range(self.n_iter):
            y_pred = X @ self.weights
            error = y_pred - y
            grad_mse = (1 / n_samples) * X.T @ error
            grad_l2 = self.alpha * (1 - self.l1_ratio) * self.weights
            grad_l1 = self.alpha * self.l1_ratio * np.sign(self.weights)
            grad = grad_mse + grad_l2 + grad_l1
            self.weights -= self.lr * grad

            mse = (1 / (2 * n_samples)) * np.sum(error ** 2)
            l2_pen = 0.5 * self.alpha * (1 - self.l1_ratio) * np.sum(self.weights ** 2)
            l1_pen = self.alpha * self.l1_ratio * np.sum(np.abs(self.weights))
            loss = mse + l2_pen + l1_pen

            if abs(prev_loss - loss) < self.tol:
                break
            prev_loss = loss

    def predict(self, X):
        return X @ self.weights

    def r2_score(y_true, y_pred):
        ss_res = np.sum((y_true - y_pred) ** 2)
        ss_tot = np.sum((y_true - np.mean(y_true)) ** 2)
        return 1 - ss_res / ss_tot

# -----
# 4. Подбор гиперпараметров (на 80% трейне) для использования во всех сценариях
# -----
np.random.seed(42)
idx = np.random.permutation(len(X))
split = int(0.8 * len(X))
train_idx, test_idx = idx[:split], idx[split:]

X_train_full, X_test = X[train_idx], X[test_idx]
y_train_full_log, y_test_orig = y_log[train_idx], y_orig[test_idx]
y_train_full_orig = y_orig[train_idx]

```

```

# Масштабирование на трейне
train_mean = X_train_full.mean(axis=0)
train_std = X_train_full.std(axis=0) + 1e-8
X_train_full_scaled = (X_train_full - train_mean) / train_std
X_test_scaled = (X_test - train_mean) / train_std

y_mean_log = y_train_full_log.mean()
y_train_full_centered = y_train_full_log - y_mean_log

# Внутренняя валидация для подбора гиперпараметров
np.random.seed(123)
val_size = int(0.2 * len(X_train_full_scaled))
sub_idx = np.random.permutation(len(X_train_full_scaled))
train_sub_idx, val_idx = sub_idx[val_size:], sub_idx[:val_size]

X_subtrain = X_train_full_scaled[train_sub_idx]
y_subtrain = y_train_full_centered[train_sub_idx]
X_val = X_train_full_scaled[val_idx]
y_val_orig = y_train_full_log[val_idx]

alphas = [0.001, 0.01, 0.05, 0.1, 0.5, 1.0]
l1_ratios = [0.1, 0.5, 0.7, 0.9, 0.99]
best_r2 = -np.inf
best_alpha, best_l1 = alphas[0], l1_ratios[0]

print("Подбор гиперпараметров на валидации...")
for alpha in alphas:
    for l1_ratio in l1_ratios:
        model = ElasticNetGD(alpha=alpha, l1_ratio=l1_ratio,
                              learning_rate=0.01, n_iter=2000, tol=1e-5)
        model.fit(X_subtrain, y_subtrain)
        y_val_pred_centered = model.predict(X_val)
        y_val_pred_log = y_val_pred_centered + y_mean_log
        y_val_pred = np.exp(y_val_pred_log)
        r2 = r2_score(y_val_orig, y_val_pred)
        if r2 > best_r2:
            best_r2 = r2
            best_alpha, best_l1 = alpha, l1_ratio

print(f"Лучшие гиперпараметры: alpha={best_alpha}, l1_ratio={best_l1}\n")

# -----
# 5. Сценарий 1: обучение и оценка на ВСЕХ данных (переобучение)
# -----
print("=== Сценарий 1: Обучение и оценка на всех данных ===")
# Масштабируем все данные
all_mean = X.mean(axis=0)
all_std = X.std(axis=0) + 1e-8
X_all_scaled = (X - all_mean) / all_std

y_all_log = y_log
y_all_centered = y_all_log - y_all_log.mean()

model_all = ElasticNetGD(alpha=best_alpha, l1_ratio=best_l1,
                          learning_rate=0.01, n_iter=3000, tol=1e-6)
model_all.fit(X_all_scaled, y_all_centered)

y_all_pred_centered = model_all.predict(X_all_scaled)
y_all_pred_log = y_all_pred_centered + y_all_log.mean()
y_all_pred = np.exp(y_all_pred_log)

r2_all = r2_score(y_orig, y_all_pred)

```



```

mae_all = np.mean(np.abs(y_orig - y_all_pred))
print(f"R² на всех данных (переобучение): {r2_all:.4f}")
print(f"MAE на всех данных: {mae_all:,.0f} руб.\n")

# -----
# 6. Сценарий 2: обычное разделение 80/20
# -----
print("=== Сценарий 2: Обычное разделение 80/20 ===")
final_model = ElasticNetGD(alpha=best_alpha, l1_ratio=best_l1,
                           learning_rate=0.01, n_iter=3000, tol=1e-6)
final_model.fit(X_train_full_scaled, y_train_full_centered)

y_test_pred_centered = final_model.predict(X_test_scaled)
y_test_pred_log = y_test_pred_centered + y_mean_log
y_test_pred = np.exp(y_test_pred_log)

r2_test = r2_score(y_test_orig, y_test_pred)
mae_test = np.mean(np.abs(y_test_orig - y_test_pred))
print(f"R² на тесте (80/20): {r2_test:.4f}")
print(f"MAE на тесте: {mae_test:,.0f} руб.\n")

# -----
# 7. Сценарий 3: кросс-валидация (5 фолдов)
# -----
print("=== Сценарий 3: 5-кратная кросс-валидация ===")
k = 5
np.random.seed(999)
indices = np.random.permutation(len(X))
fold_size = len(X) // k
r2_scores = []

for fold in range(k):
    # Определяем индексы текущего фолда
    start = fold * fold_size
    end = start + fold_size if fold != k-1 else len(X)
    val_idx_cv = indices[start:end]
    train_idx_cv = np.setdiff1d(indices, val_idx_cv)

    X_train_cv, X_val_cv = X[train_idx_cv], X[val_idx_cv]
    y_train_cv_log, y_val_cv_orig = y_log[train_idx_cv], y_orig[val_idx_cv]

    # Масштабирование только по обучающей части фолда
    mu = X_train_cv.mean(axis=0)
    sigma = X_train_cv.std(axis=0) + 1e-8
    X_train_cv_scaled = (X_train_cv - mu) / sigma
    X_val_cv_scaled = (X_val_cv - mu) / sigma

    y_train_cv_centered = y_train_cv_log - y_train_cv_log.mean()

    # Обучение модели с теми же гиперпараметрами
    model_cv = ElasticNetGD(alpha=best_alpha, l1_ratio=best_l1,
                           learning_rate=0.01, n_iter=2000, tol=1e-5)
    model_cv.fit(X_train_cv_scaled, y_train_cv_centered)

    # Предсказание и перевод в исходный масштаб
    y_val_pred_centered = model_cv.predict(X_val_cv_scaled)
    y_val_pred_log = y_val_pred_centered + y_train_cv_log.mean()
    y_val_pred = np.exp(y_val_pred_log)

    r2_fold = r2_score(y_val_cv_orig, y_val_pred)
    r2_scores.append(r2_fold)
    print(f"Фолд {fold+1}: R² = {r2_fold:.4f}")

```

```

r2_mean = np.mean(r2_scores)
r2_std = np.std(r2_scores)
print(f"\nСреднее R2 по кросс-валидации: {r2_mean:.4f} ± {r2_std:.4f}")

```

ElasticNet Scikit

```

import pandas as pd
import numpy as np
from sklearn.linear_model import ElasticNetCV, ElasticNet
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import r2_score, mean_absolute_error

# -----
# 1. Загрузка и подготовка данных
# -----
data = pd.read_excel('ml_moscow_flats.xlsx')
data = data.drop_duplicates().reset_index(drop=True)

# Исправление десятичных запятых
for col in ['latitude', 'longitude', 'totalArea', 'kitchenArea']:
    if data[col].dtype == object:
        data[col] = data[col].astype(str).str.replace(',', '.').astype(float)

# -----
# 2. Генерация признаков
# -----
def haversine_distance(lat1, lon1, lat2, lon2):
    R = 6371
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat/2)**2 + np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2)**2
    c = 2 * np.arcsin(np.sqrt(a))
    return R * c

center_lat, center_lon = 55.751244, 37.618423
data['dist_to_center'] = data.apply(
    lambda row: haversine_distance(row['latitude'], row['longitude'], center_lat, center_lon),
    axis=1)

data['floor_ratio'] = data['floorNumber'] / data['floorsTotal']
data['is_first_floor'] = (data['floorNumber'] == 1).astype(int)
data['is_last_floor'] = (data['floorNumber'] == data['floorsTotal']).astype(int)
data['kitchen_area_ratio'] = data['kitchenArea'] / data['totalArea']
data['total_area_sq'] = data['totalArea'] ** 2
data['log_total_area'] = np.log(data['totalArea'])
data['log_kitchen_area'] = np.log(data['kitchenArea'] + 1)

walls_dummies = pd.get_dummies(data['wallsMaterial'], prefix='walls')
data = pd.concat([data, walls_dummies], axis=1)

features = [
    'floorNumber', 'floorsTotal', 'totalArea', 'kitchenArea',
    'dist_to_center', 'floor_ratio', 'is_first_floor', 'is_last_floor',
    'kitchen_area_ratio', 'total_area_sq', 'log_total_area', 'log_kitchen_area'
] + list(walls_dummies.columns)

X = data[features].values.astype(np.float64)

```

```

y = data['price'].values.astype(np.float64)

# Логарифмирование целевой переменной
y_log = np.log(y)

# -----
# 3. Масштабирование признаков
# -----
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# -----
# 4. Сценарий 1: обучение и оценка на всех данных
# -----
print("=== Сценарий 1: Все данные ===")
# Подбор гиперпараметров на всех данных с помощью кросс-валидации
cv_model_all = ElasticNetCV(l1_ratio=[0.1, 0.5, 0.7, 0.9, 0.99],
                             alphas=[0.001, 0.01, 0.05, 0.1, 0.5, 1.0],
                             cv=5, random_state=42, max_iter=5000)
cv_model_all.fit(X_scaled, y_log)

best_alpha = cv_model_all.alpha_
best_l1 = cv_model_all.l1_ratio_
print(f"Лучшие параметры: alpha={best_alpha:.4f}, l1_ratio={best_l1:.2f}")

# Обучение финальной модели на всех данных
model_all = ElasticNet(alpha=best_alpha, l1_ratio=best_l1, max_iter=5000)
model_all.fit(X_scaled, y_log)

# Предсказание и перевод в исходный масштаб
y_pred_log_all = model_all.predict(X_scaled)
y_pred_all = np.exp(y_pred_log_all)

r2_all = r2_score(y, y_pred_all)
mae_all = mean_absolute_error(y, y_pred_all)
print(f"R² на всех данных: {r2_all:.4f}")
print(f"MAE на всех данных: {mae_all:.0f} руб.\n")

# -----
# 5. Сценарий 2: разделение 80/20
# -----
print("=== Сценарий 2: Разделение 80/20 ===")
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_log, test_size=0.2, random_state=42)

# Подбор параметров только на обучающей части
cv_model_8020 = ElasticNetCV(l1_ratio=[0.1, 0.5, 0.7, 0.9, 0.99],
                              alphas=[0.001, 0.01, 0.05, 0.1, 0.5, 1.0],
                              cv=5, random_state=42, max_iter=5000)
cv_model_8020.fit(X_train, y_train)

best_alpha_80 = cv_model_8020.alpha_
best_l1_80 = cv_model_8020.l1_ratio_
print(f"Лучшие параметры: alpha={best_alpha_80:.4f}, l1_ratio={best_l1_80:.2f}")

model_8020 = ElasticNet(alpha=best_alpha_80, l1_ratio=best_l1_80, max_iter=5000)
model_8020.fit(X_train, y_train)

y_pred_log_test = model_8020.predict(X_test)
y_pred_test = np.exp(y_pred_log_test)

r2_test = r2_score(np.exp(y_test), y_pred_test)

```

```

mae_test = mean_absolute_error(np.exp(y_test), y_pred_test)
print(f'R² на тесте: {r2_test:.4f}')
print(f'MAE на тесте: {mae_test:,.0f} руб.\n")

# -----
# 6. Сценарий 3: 5-кратная кросс-валидация
# -----
print("=== Сценарий 3: Кросс-валидация (5 фолдов) ===")
# Для кросс-валидации используем ElasticNetCV, чтобы внутри каждого фолда
# параметры подбирались заново – это даёт более честную оценку.
cv_scores = cross_val_score(
    ElasticNetCV(l1_ratio=[0.1, 0.5, 0.7, 0.9, 0.99],
                 alphas=[0.001, 0.01, 0.05, 0.1, 0.5, 1.0],
                 cv=3, max_iter=5000, random_state=42),
    X_scaled, y_log, cv=5, scoring='r2'
)

print(f'R² по фолдам: {cv_scores}')
print(f'Среднее R²: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}')

```

MLP самописный

```

import numpy as np
import pandas as pd

# ----- Вспомогательные классы -----
class OneHotEncoder:
    def fit(self, col):
        vals = np.array(col).astype(str)
        self.categories_ = np.unique(vals)
        return self
    def transform(self, col):
        vals = np.array(col).astype(str)
        mat = np.zeros((len(vals), len(self.categories_)))
        for i, cat in enumerate(self.categories_):
            mat[:, i] = (vals == cat).astype(int)
        return mat
    def fit_transform(self, col):
        self.fit(col)
        return self.transform(col)

class StandardScaler:
    def fit(self, X):
        self.mean = X.mean(axis=0, keepdims=True)
        self.std = X.std(axis=0, keepdims=True) + 1e-8
        return self
    def transform(self, X):
        return (X - self.mean) / self.std
    def fit_transform(self, X):
        self.fit(X)
        return self.transform(X)

def safe_float(series):
    return series.astype(str).str.replace(',', '.').astype(float)

# ----- MLPRegressor (чистый NumPy) -----
class MLPRegressor:
    def __init__(self, hidden_layers=(64,), activation='relu', lr=1e-3,
                 epochs=1000, batch_size=32, val_split=0.1, patience=20,
                 weight_decay=1e-4, verbose=False):

```

```

self.hidden_layers = hidden_layers
self.activation_name = activation
self.lr = lr
self.epochs = epochs
self.batch_size = batch_size
self.val_split = val_split
self.patience = patience
self.weight_decay = weight_decay
self.verbose = verbose

def _init_weights(self, layer_dims):
    self.W, self.b = [], []
    for i in range(len(layer_dims)-1):
        fan_in = layer_dims[i]
        fan_out = layer_dims[i+1]
        std = np.sqrt(2./fan_in) if self.activation_name == 'relu' else np.sqrt(1./fan_in)
        self.W.append(np.random.randn(fan_in, fan_out)*std)
        self.b.append(np.zeros((1, fan_out)))

def _activate(self, Z):
    if self.activation_name == 'relu': return np.maximum(0, Z)
    if self.activation_name == 'tanh': return np.tanh(Z)
    return Z

def _activate_derivative(self, A):
    if self.activation_name == 'relu': return (A > 0).astype(float)
    if self.activation_name == 'tanh': return 1 - A**2
    return np.ones_like(A)

def _forward(self, X):
    self.Z, self.A = [], [X]
    for i in range(len(self.W)):
        z = self.A[-1] @ self.W[i] + self.b[i]
        self.Z.append(z)
        a = z if i == len(self.W)-1 else self._activate(z)
        self.A.append(a)
    return self.A[-1]

def _backward(self, y_true, y_pred):
    m = y_true.shape[0]
    dA = (y_pred - y_true) / m
    grads_W, grads_b = [], []
    for i in reversed(range(len(self.W))):
        dZ = dA if i == len(self.W)-1 else dA * self._activate_derivative(self.A[i+1])
        dW = self.A[i].T @ dZ + self.weight_decay * self.W[i]
        db = np.sum(dZ, axis=0, keepdims=True)
        dA = dZ @ self.W[i].T
        grads_W.insert(0, dW)
        grads_b.insert(0, db)
    return grads_W, grads_b

def _adam_init(self):
    self.m_W = [np.zeros_like(w) for w in self.W]
    self.v_W = [np.zeros_like(w) for w in self.W]
    self.m_b = [np.zeros_like(b) for b in self.b]
    self.v_b = [np.zeros_like(b) for b in self.b]
    self.t = 0

def _adam_update(self, grads_W, grads_b):
    self.t += 1
    beta1, beta2, eps = 0.9, 0.999, 1e-8
    for i in range(len(self.W)):

```

```

self.m_W[i] = beta1*self.m_W[i] + (1-beta1)*grads_W[i]
self.v_W[i] = beta2*self.v_W[i] + (1-beta2)*grads_W[i]**2
m_hat = self.m_W[i] / (1 - beta1**self.t)
v_hat = self.v_W[i] / (1 - beta2**self.t)
self.W[i] -= self.lr * m_hat / (np.sqrt(v_hat)+eps)

```

```

self.m_b[i] = beta1*self.m_b[i] + (1-beta1)*grads_b[i]
self.v_b[i] = beta2*self.v_b[i] + (1-beta2)*grads_b[i]**2
m_hat_b = self.m_b[i] / (1 - beta1**self.t)
v_hat_b = self.v_b[i] / (1 - beta2**self.t)
self.b[i] -= self.lr * m_hat_b / (np.sqrt(v_hat_b)+eps)

```

```

def fit(self, X, y, X_val=None, y_val=None):

```

```

    """
    Если X_val и y_val не переданы, валидационная часть берётся из X, y
    согласно val_split. Если val_split=0, валидация отсутствует.
    """

```

```

    if X_val is None and self.val_split > 0:

```

```

        np.random.seed(42)
        idx = np.random.permutation(len(X))
        val_size = int(len(X)*self.val_split)
        train_idx, val_idx = idx[val_size:], idx[:val_size]
        X_train, y_train = X[train_idx], y[train_idx]
        X_val, y_val = X[val_idx], y[val_idx]

```

```

    else:

```

```

        X_train, y_train = X, y

```

```

    layer_dims = [X.shape[1]] + list(self.hidden_layers) + [1]

```

```

    self._init_weights(layer_dims)

```

```

    self._adam_init()

```

```

    best_loss = np.inf

```

```

    wait = 0

```

```

    for epoch in range(self.epochs):

```

```

        perm = np.random.permutation(len(X_train))

```

```

        X_train, y_train = X_train[perm], y_train[perm]

```

```

        for i in range(0, len(X_train), self.batch_size):

```

```

            Xb, yb = X_train[i:i+self.batch_size], y_train[i:i+self.batch_size]

```

```

            y_pred = self._forward(Xb)

```

```

            grads_W, grads_b = self._backward(yb, y_pred)

```

```

            self._adam_update(grads_W, grads_b)

```

```

        # оценка на валидации (если есть)

```

```

        if X_val is not None:

```

```

            val_pred = self._forward(X_val)

```

```

            val_loss = np.mean((val_pred - y_val)**2)

```

```

            if self.verbose and epoch%100==0:

```

```

                print(f" Epoch {epoch}: val_loss={val_loss:.6f}")

```

```

            if val_loss < best_loss:

```

```

                best_loss = val_loss

```

```

                wait = 0

```

```

                best_W, best_b = [w.copy() for w in self.W], [b.copy() for b in self.b]

```

```

            else:

```

```

                wait += 1

```

```

                if wait >= self.patience:

```

```

                    if self.verbose: print(f" Early stop at epoch {epoch}")

```

```

                    break

```

```

        else:

```

```

            # без валидации просто запоминаем последние веса

```

```

            best_W, best_b = [w.copy() for w in self.W], [b.copy() for b in self.b]

```

```

    self.W, self.b = best_W, best_b

```

```

def predict(self, X):
    return self._forward(X)

# ----- Загрузка и подготовка сырых данных -----
df = pd.read_excel("ml_moscow_flats.xlsx")
df = df.dropna(subset=["price"])

# Исходные столбцы
walls_raw = df["wallsMaterial"].fillna('unknown').astype(str).values
num_features = {
    "floorNumber": safe_float(df["floorNumber"].values.reshape(-1,1),
    "floorsTotal": safe_float(df["floorsTotal"].values.reshape(-1,1),
    "totalArea": safe_float(df["totalArea"].values.reshape(-1,1),
    "kitchenArea": safe_float(df["kitchenArea"].values.reshape(-1,1),
    "latitude": safe_float(df["latitude"].values.reshape(-1,1),
    "longitude": safe_float(df["longitude"].values.reshape(-1,1)
}
y_log = np.log(df["price"].values.reshape(-1,1).astype(float) + 1e-8)

# ----- Метрика R² -----
def r2_score(y_true, y_pred):
    ss_res = np.sum((y_true - y_pred)**2)
    ss_tot = np.sum((y_true - np.mean(y_true))**2)
    return 1 - ss_res/ss_tot

# ----- Сценарий 1: обучение = проверка (все данные) -----
print("=*60)
print("СЦЕНАРИЙ 1: ОБУЧЕНИЕ И ПРОВЕРКА НА ВСЕХ ДАННЫХ (переобучение)")
# one-hot и масштабирование на всех данных
ohe_all = OneHotEncoder()
walls_ohe = ohe_all.fit_transform(walls_raw)
num_all = np.hstack([num_features[k] for k in num_features])
scaler_all = StandardScaler()
num_scaled = scaler_all.fit_transform(num_all)
X_all = np.hstack([walls_ohe, num_scaled])
y_all = y_log

mlp_all = MLPRegressor(hidden_layers=[128,64], activation='relu', lr=0.001,
                        epochs=1000, batch_size=64, val_split=0.0, # валидация отключена
                        patience=500, weight_decay=1e-4, verbose=True)
mlp_all.fit(X_all, y_all) # нет валидации
pred_all = mlp_all.predict(X_all)
r2_train = r2_score(np.exp(y_all)-1e-8, np.exp(pred_all)-1e-8)
print(f"R² на всех (тренировочных) данных: {r2_train:.4f}\n")

# ----- Сценарий 2: Обычное разбиение 80/20 -----
print("=*60)
print("СЦЕНАРИЙ 2: ОБЫЧНОЕ РАЗБИЕНИЕ 80% train / 20% test")
np.random.seed(42)
idx = np.random.permutation(len(df))
split = int(0.8 * len(df))
train_idx, test_idx = idx[:split], idx[split:]

ohe = OneHotEncoder()
X_train_walls = ohe.fit_transform(walls_raw[train_idx])
X_test_walls = ohe.transform(walls_raw[test_idx])

num_train = np.hstack([num_features[k][train_idx] for k in num_features])
num_test = np.hstack([num_features[k][test_idx] for k in num_features])

scaler = StandardScaler()

```

```

X_train_num = scaler.fit_transform(num_train)
X_test_num = scaler.transform(num_test)

X_train = np.hstack([X_train_walls, X_train_num])
X_test = np.hstack([X_test_walls, X_test_num])
y_train, y_test = y_log[train_idx], y_log[test_idx]

mlp2 = MLPRegressor(hidden_layers=[128,64], activation='relu', lr=0.001,
                    epochs=5000, batch_size=64, val_split=0.1, patience=50,
                    weight_decay=1e-4, verbose=True)
mlp2.fit(X_train, y_train)
pred_test = mlp2.predict(X_test)
r2_holdout = r2_score(np.exp(y_test)-1e-8, np.exp(pred_test)-1e-8)
print(f"R² на тестовой выборке: {r2_holdout:.4f}\n")

# ----- Сценарий 3: Кросс-валидация (5 фолдов) -----
print("="*60)
print("СЦЕНАРИЙ 3: КРОСС-ВАЛИДАЦИЯ (5 фолдов)")
k = 5
np.random.seed(42)
indices = np.random.permutation(len(df))
fold_size = len(df) // k
folds = [indices[i*fold_size:(i+1)*fold_size] for i in range(k-1)]
folds.append(indices[(k-1)*fold_size:])

r2_cv = []
for fold in range(k):
    test_idx = folds[fold]
    train_idx = np.concatenate([folds[i] for i in range(k) if i != fold])

    ohe_cv = OneHotEncoder()
    X_tr_w = ohe_cv.fit_transform(walls_raw[train_idx])
    X_te_w = ohe_cv.transform(walls_raw[test_idx])

    num_tr = np.hstack([num_features[k][train_idx] for k in num_features])
    num_te = np.hstack([num_features[k][test_idx] for k in num_features])

    scaler_cv = StandardScaler()
    X_tr_num = scaler_cv.fit_transform(num_tr)
    X_te_num = scaler_cv.transform(num_te)

    X_tr = np.hstack([X_tr_w, X_tr_num])
    X_te = np.hstack([X_te_w, X_te_num])
    y_tr, y_te = y_log[train_idx], y_log[test_idx]

    print(f"\nFold {fold+1}/{k}")
    mlp_cv = MLPRegressor(hidden_layers=[128,64], activation='relu', lr=0.001,
                        epochs=5000, batch_size=64, val_split=0.1, patience=50,
                        weight_decay=1e-4, verbose=False)
    mlp_cv.fit(X_tr, y_tr)
    pred = mlp_cv.predict(X_te)
    r2_fold = r2_score(np.exp(y_te)-1e-8, np.exp(pred)-1e-8)
    r2_cv.append(r2_fold)
    print(f"R² fold {fold+1}: {r2_fold:.4f}")

print("\n" + "="*60)
print(f"Среднее R² (кросс-валидация): {np.mean(r2_cv):.4f} ± {np.std(r2_cv):.4f}")
print(f"Сравнение:")
print(f" - R² на всех данных (переобучение): {r2_train:.4f}")
print(f" - R² на тесте (80/20): {r2_holdout:.4f}")
print(f" - R² среднее по CV: {np.mean(r2_cv):.4f}")

```


MLP scikit-learn

```
import numpy as np
import pandas as pd
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import r2_score

# ----- Загрузка и очистка данных -----
df = pd.read_excel("ml_moscow_flats.xlsx").dropna(subset=["price"])

# Числовые колонки: заменяем запятые на точки, конвертируем в float
num_cols = ["floorNumber", "floorsTotal", "totalArea", "kitchenArea",
            "latitude", "longitude"]
for col in num_cols:
    df[col] = df[col].astype(str).str.replace(',', '.').astype(float)

# Категориальная колонка
cat_col = "wallsMaterial"
df[cat_col] = df[cat_col].fillna('unknown').astype(str)

# Целевая переменная с логарифмированием
y = np.log(df["price"].values + 1e-8)

# Признаки: правильное объединение списка имён колонок
feature_cols = [cat_col] + num_cols
X = df[feature_cols]

# ----- Сценарий 1: все данные (переобучение) -----
print("=" * 60)
print("СЦЕНАРИЙ 1: ОБУЧЕНИЕ НА ВСЕХ ДАННЫХ, ПРОВЕРКА НА НИХ ЖЕ")
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), [cat_col]),
        ("num", StandardScaler(), num_cols)
    ]
)
X_all = preprocessor.fit_transform(X)

mlp1 = MLPRegressor(hidden_layer_sizes=(128, 64), activation='relu',
                    solver='adam', alpha=1e-4, batch_size=64,
                    learning_rate_init=0.001, max_iter=5000,
                    early_stopping=False, random_state=42)
mlp1.fit(X_all, y)
pred1 = mlp1.predict(X_all)
r2_train = r2_score(np.exp(y) - 1e-8, np.exp(pred1) - 1e-8)
print(f"R² на всех (тренировочных) данных: {r2_train:.4f}\n")

# ----- Сценарий 2: обычное разбиение 80/20 -----
print("=" * 60)
print("СЦЕНАРИЙ 2: ОБЫЧНОЕ РАЗБИЕНИЕ 80% train / 20% test")
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

preprocessor2 = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), [cat_col]),
        ("num", StandardScaler(), num_cols)
    ]
)
X_train_prep = preprocessor2.fit_transform(X_train)
X_test_prep = preprocessor2.transform(X_test)
```

```

mlp2 = MLPRegressor(hidden_layer_sizes=(128, 64), activation='relu',
                    solver='adam', alpha=1e-4, batch_size=64,
                    learning_rate_init=0.001, max_iter=5000,
                    early_stopping=True, validation_fraction=0.1,
                    n_iter_no_change=50, random_state=42)
mlp2.fit(X_train_prep, y_train)
pred2 = mlp2.predict(X_test_prep)
r2_holdout = r2_score(np.exp(y_test) - 1e-8, np.exp(pred2) - 1e-8)
print(f"R² на тестовой выборке: {r2_holdout:.4f}\n")

# ----- Сценарий 3: кросс-валидация (5 фолдов) -----
print("=" * 60)
print("СЦЕНАРИЙ 3: КРОСС-ВАЛИДАЦИЯ (5 фолдов)")
pipeline = Pipeline(steps=[
    ("preprocessor", ColumnTransformer(
        transformers=[
            ("cat", OneHotEncoder(handle_unknown="ignore"), [cat_col]),
            ("num", StandardScaler(), num_cols)
        ]),
    ("mlp", MLPRegressor(hidden_layer_sizes=(128, 64), activation='relu',
                        solver='adam', alpha=1e-4, batch_size=64,
                        learning_rate_init=0.001, max_iter=5000,
                        early_stopping=True, validation_fraction=0.1,
                        n_iter_no_change=50, random_state=42))
])

cv = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(pipeline, X, y, cv=cv, scoring='r2')
print(f"R² по фолдам: {scores}")
print(f"Среднее R² (CV): {np.mean(scores):.4f} ± {np.std(scores):.4f}")

print("\n" + "=" * 60)
print(f"Сравнение:")
print(f"- Переобучение (все данные): R² = {r2_train:.4f}")
print(f"- Hold-out 80/20: R² = {r2_holdout:.4f}")
print(f"- Кросс-валидация: R² = {np.mean(scores):.4f} ± {np.std(scores):.4f}")

```